

The virtual operational brain for enterprises that orchestrates multi-agent AI to analyze collaboration patterns, predict project bottlenecks, and autonomously coordinate business workflows.

"The virtual operational brain for enterprises that orchestrates multi-agent AI to analyze collaboration patterns, predict project bottlenecks, and autonomously coordinate business workflows."

BLUEPRINT SPECIFICATIONS

Original Prompt:	An advanced AI-powered cognitive workflow intelligence platform that continuously analyzes organizational communication, project activities, operational patterns,...
Target Tech:	Vite + React + Tailwind CSS + Node.js + Express.js + PostgreSQL + FastAPI + Redis + Docker + Amazon Web Services
Engine Model:	gemini-3.5-flash-guided
Architect E-mail:	chavatasiva@gmail.com
Generated Date:	May 26, 2026

DOCUMENT CONTENTS

- 01 Product Requirements Document (PRD)**
Core business constraints, scopes & features
- 02 Sprint Backlog & User Stories**
User requirements and point estimates mapped over sprints
- 03 Development Milestone Roadmap**
Sequence checklists, goals & readiness criteria

01. PRODUCT REQUIREMENTS DOCUMENT (PRD)

Product Requirements Document (PRD)

Project: AetherOS - Enterprise Cognitive Workflow Intelligence Platform

Document Version: 1.0.0

Author: Principal Product Manager & Systems Architect

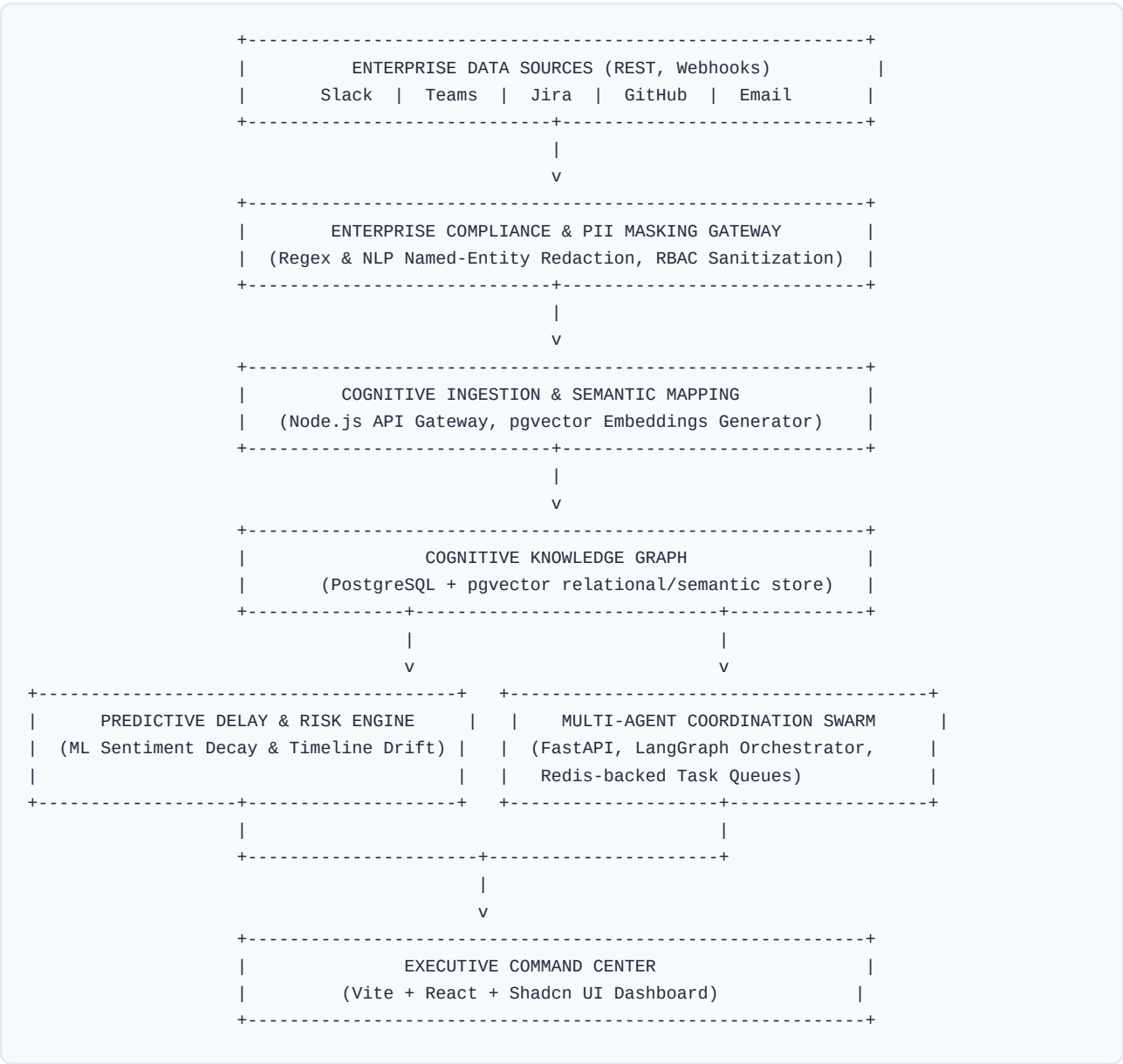
Status: Ready for Architecture Review

1. Executive Summary & Core Definitions

1.1 Executive Summary

Modern enterprises operate in environments characterized by hyper-fragmented communication channels, disconnected project tracking software, and overwhelming operational noise. Valuable structural signal is lost in the gaps between Slack messages, Jira tickets, GitHub pull requests, and corporate emails.

AetherOS is an advanced AI-powered cognitive workflow intelligence platform that serves as a real-time "virtual operational brain" for enterprises. By continuously ingestion-mapping disparate collaboration patterns, AetherOS constructs a dynamic **Cognitive Knowledge Graph** of the organization. Leveraging a secure, multi-agent AI swarm, the platform proactively predicts delivery bottlenecks, automates cross-functional coordination, and delivers high-fidelity strategic insights directly to leaders, transforming passive operational exhaust into an active, self-healing workflow engine.



1.2 Core Definitions & Glossary

- **Cognitive Knowledge Graph (CKG):** A unified relational and vector database structure representing organizational entities (employees, teams, tasks, repositories, communications) and the dynamic, semantic relationships between them.
- **Multi-Agent Swarm:** A coordinated system of specialized LLM-powered agents (e.g., Triage, Coordination, Risk, Governance) working collaboratively using defined protocols to execute tasks without human intervention.
- **PII Masking Gateway:** A high-throughput, deterministic, and NLP-driven security layer that intercepts all outgoing LLM payloads to redact Personally Identifiable Information (PII) and Intellectual Property (IP) before processing.
- **Drift Coefficient:** An analytical score calculated by the platform indicating the statistical probability that a project schedule will slip based on historical cycle times, communication decay, and active dependencies.

- ****Contextual Memory Cascade:**** A multi-tiered caching mechanism (Redis L1 -> Postgres L2 -> Vector L3) allowing agents to maintain persistent execution context over long-running asynchronous workflows.
-

2. Product Vision, Goals, and Non-Goals

2.1 Product Vision

To build a frictionless, autonomous operational layer that transforms enterprise execution from reactive crisis management into proactive, self-orchestrating delivery.

2.2 Goals (MVP Phase)

- ****Zero-Config Cognitive Mapping:**** Establish a real-time, bi-directional ingestion loop with Slack, Microsoft Teams, Jira, and GitHub within 15 minutes of enterprise installation.
- ****High-Fidelity Delay Prediction:**** Predict software release and project delivery delays with $>85\%$ accuracy up to 14 days before an official milestone breach occurs.
- ****Autonomous Coordination Loop:**** Automate at least 40% of standard, manual cross-team update collection and ticket-advancement tasks using secure multi-agent collaboration.
- ****Enterprise-Grade Security First:**** Enforce a zero-leakage policy for PII, proprietary source code, and internal corporate secrets at the LLM ingestion boundary.
- ****Low-Code Adjustability:**** Provide operations leaders with a drag-and-drop workspace to alter agent operational parameters, execution thresholds, and communication boundaries.

2.3 Non-Goals (Out of Scope for MVP)

- ****Direct Replacement of Productivity Tooling:**** AetherOS does not aim to replace Slack, Jira, or GitHub. It operates strictly as an intelligent Orchestration Layer sitting *on top* of them.
 - ****Employee Performance Evaluation:**** AetherOS **will not** generate individual productivity rankings, surveillance metrics, or "active/idle" monitoring indicators. This is a critical architectural decision to ensure trust, ethical compliance, and high adoption rates.
 - ****Automated Code Synthesis or Refactoring:**** While the agent swarm can analyze pull request patterns, it will not write, edit, or commit source code directly to repositories.
 - ****On-Premises Air-Gapped Deployment:**** The initial release will target an Enterprise-grade Dedicated Virtual Private Cloud (VPC) deployment model on AWS rather than localized bare-metal setups.
-

3. User Personas & User Journeys

3.1 User Personas

Persona A: Sarah Jenkins - Chief Operating Officer (COO)

- **Role Context:** Manages cross-functional execution across a 2,500-person digital enterprise. Oversees PMO, delivery, and internal operational tooling.
- **Core Pain Points:**
 - No single source of truth for cross-functional project health; relies on heavily biased, manual, and outdated weekly status reports.
 - Silos between engineering, product, and marketing lead to delayed alignment and missed product launch dates.
 - Wastes hours in sync meetings resolving miscommunications.
- **Platform Goals:** Wants a real-time, unbiased, automated "operational weather map" of the business that can self-correct basic coordination bottlenecks without her direct intervention.

Persona B: David Chen - PMO & Agile Delivery Director

- **Role Context:** Leads a team of 15 Program Managers and Scrum Masters coordinating complex software release cycles.
- **Core Pain Points:**
 - Spends 60% of his day hunting down project updates, reminding engineers to update tickets, and manually linking Jira issues across departments.
 - Cannot identify hidden upstream dependencies (e.g., a delayed API design blocking three downstream front-end tracks) until the target release day.
- **Platform Goals:** Wants to automate mundane progress-chasing tasks and receive early automated alerts pointing to the specific, high-risk points of structural drift.

3.2 User Journeys

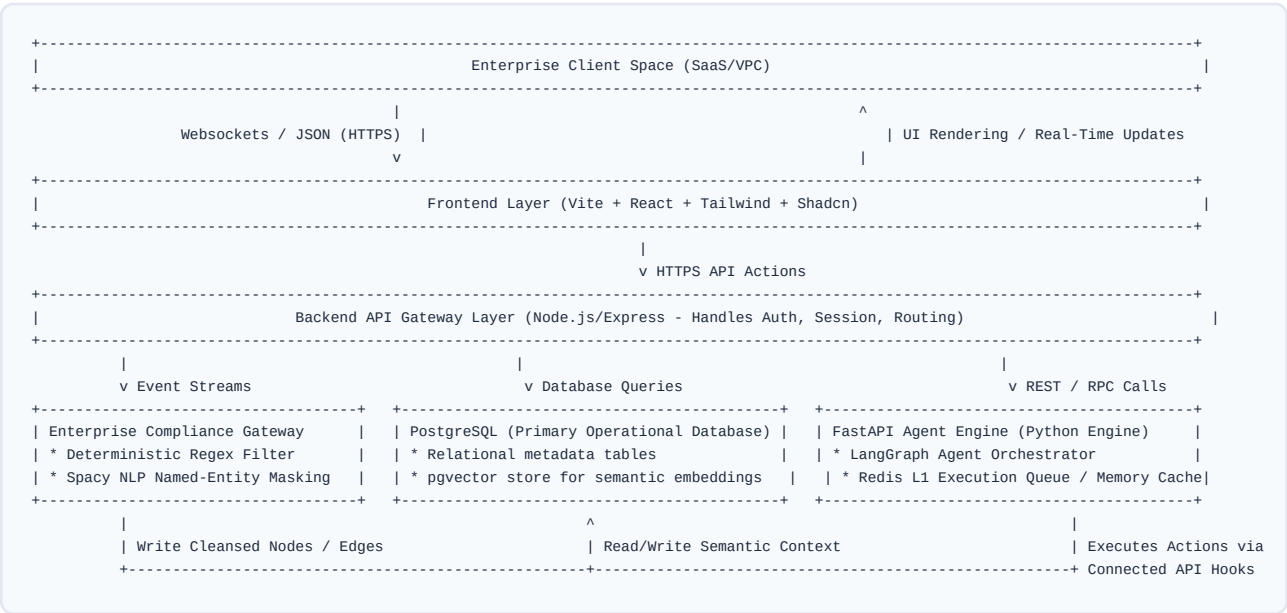
Journey 1: Identifying and Defusing a Critical Path Delay (Predictive Alerting to Autonomous Resolution)

[System Identifies Risk] ---> [Alert Sent to PMO] ---> [Agent Swarm Initiates Sync] ---> [Jira Updated & Resolved]
Jira tasks stagnate & Slack sentiment drops. David notified of high Drift Coeff. Slack thread gathers info & drafts actionable remediation. Correct dependencies mapped; ticket updated.

1. **Detection:** The **Predictive Delay & Risk Engine** detects a drop in collaboration velocity on a critical Jira Epic ("Data Pipeline v2"). The sentiment score in the Slack channel `#data-infra-sync` has decayed by 34% over 48 hours, and associated GitHub PRs have been languishing with unresolved review comments.
 2. **Notification:** David (PMO Director) logs into AetherOS. The **Executive Command Center** highlights the "Data Pipeline v2" project with a 92% Drift Coefficient, predicting a 12-day launch delay.
 3. **Autonomous Swarm Mobilization:** With David's one-click approval, the **Multi-Agent Coordination Orchestrator** initializes a focused triage sweep:
 - **Triage Agent** constructs a brief, context-aware query.
 - **Coordination Agent** reaches out directly to the Lead Platform Architect via Slack, asking if they are blocked by the pending Security Review.
 - The Architect responds: "Yes, we need sign-off on the schema from Security."
 4. **Action Orchestration:** The **Coordination Agent** locates the Security Review ticket in Jira, identifies the on-duty Security Engineer via the CKG, pings them with a summary of the blocker, and dynamically logs this structural dependency into the **Cognitive Knowledge Graph**.
 5. **Resolution:** Within 2 hours, the security review is cleared. The platform automatically updates the blocker status in Jira, notifies the platform team, and recalculates the Drift Coefficient down to 4%. David receives a push notification confirming the successful auto-resolution.
-

4. Architectural & System Design Overview

To support real-time ingestion, continuous analytical evaluations, and immediate automated agent responses within a highly secure enterprise boundary, AetherOS utilizes a hybrid microservices framework designed for scalability and high performance.



4.1 System Topology

- Frontend Layer:** A single-page application built using **Vite, React, Tailwind CSS**, and **Shadcn UI**. Uses **Recharts** for visualizing delivery timelines, risk vectors, and multi-agent interaction sequences. Communicates with the API Gateway via secure WebSockets and HTTPS.
- Backend API Gateway (Node.js + Express):** Handles client-facing requests, serves as the gatekeeper for user authentication, coordinates session management, and schedules high-throughput data sync jobs.
- FastAPI Multi-Agent Engine:** A dedicated Python microservice designed to host the multi-agent LLM coordination frameworks (e.g., using **LangGraph** or **CrewAI** structures). It connects with Redis for distributed task queuing and runs the dynamic contextual memory systems.
- Database Layer (PostgreSQL with `pgvector`):** Contains the primary operational relational schema along with vector representations of ingested communication fragments, tasks, and commits to support rapid, low-latency semantic semantic searches.
- Data Isolation & Compliance Boundary (PII Masking Gateway):** Built directly into the ingestion and outbound egress pipeline. Relies on deterministic string verification alongside a local SpaCy NER model to scrubbing sensitive customer records prior to transferring metrics to the foundational model services.

5. Functional Requirements

5.1 Cognitive Ingestion & Semantic Graph Mapping (CI)

CONFIDENTIAL SPRINT DELIVERABLE MAPPED FOR: THE VIRTUAL OPERATIONAL BRAIN FOR ENT... Sheet 8 of 28

CI-1.2	Entity Resolution & Mapping	Parse raw payloads to identify distinct Entities (People, Tasks, Channels, PRs, Commits) and generate clean relationships (e.g., Person A commented on Task B).	P0	High	All	High: Resolves identity fragmentation (e.g., aligning a Git author email with a Slack workspace ID).	Build a unified mapping layer utilizing deterministic identity matching (SSO / Email mapping) to join cross-platform accounts.
CI-1.3	Semantic Embedding Generator	Automatically generate dense vector representations (using `text-embedding-3-small` or an equivalent offline local model) for unstructured communication blocks and ticket descriptions.	P0	High	PMO, COO	High: Enables the semantic layer to find contextual similarities between distinct communication streams.	Index all incoming raw textual payloads >30\$ characters. Store vectors dynamically into `pgvector` indexed column sets.
CI-1.4	Historical Ingestion Pipeline	A batch-processing engine designed to sync the prior 90 days of project, code, and communication logs on demand during initial onboarding.	P1	Medium	PMO, Admin	Medium: Prevents a cold-start problem, giving the Risk Engine immediate predictive capability.	Limit initial sync to the top 25 active Jira boards and top 10 Slack channels configured during setup.

5.2 Predictive Delay & Risk Engine (PD)

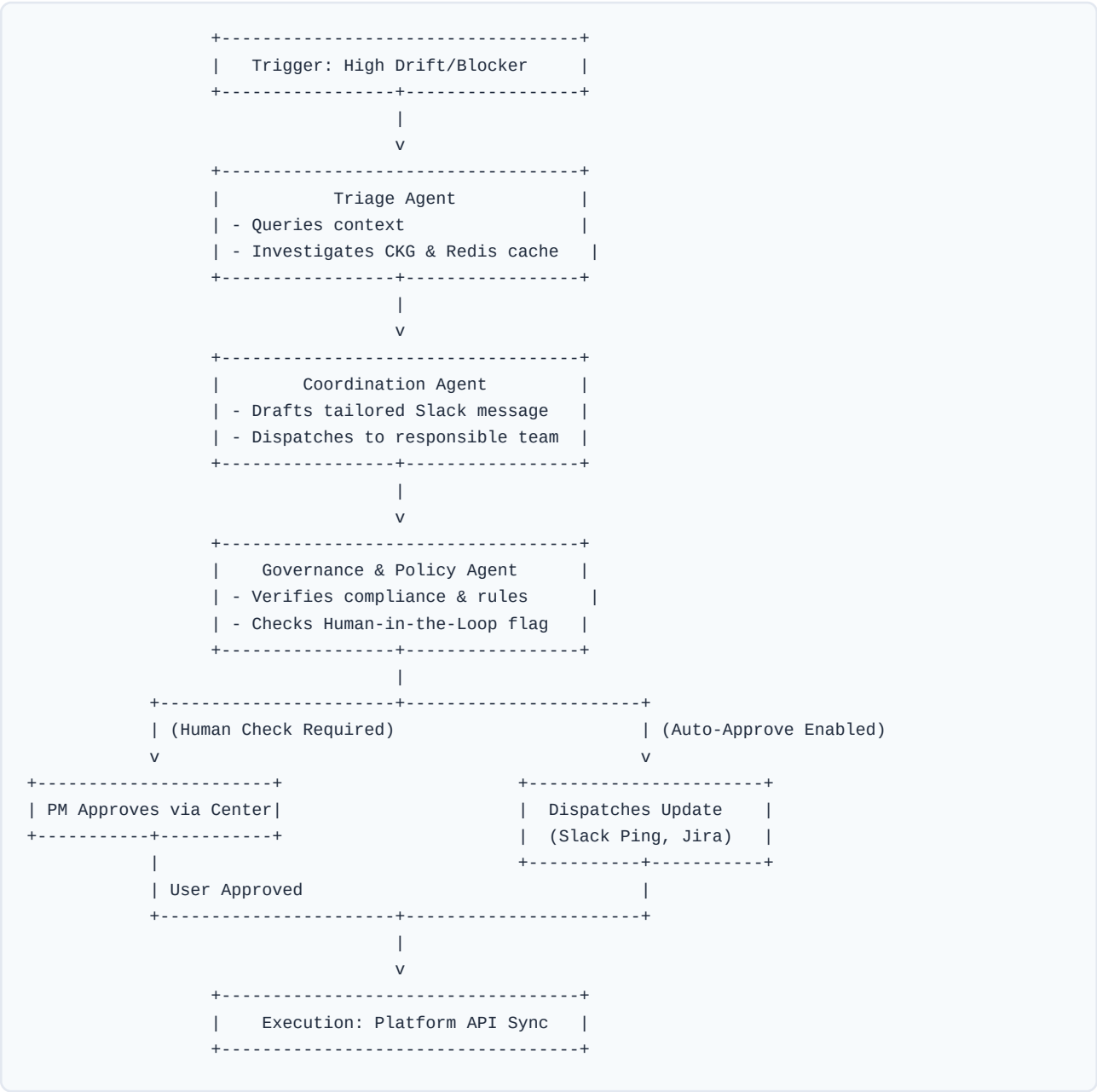
The analytical core of AetherOS. This service analyzes delivery velocity, evaluates organizational friction, and measures context fragmentation to detect milestones on track for failure.

ID	Name	Description	Priority	Complexity	Target Users	Impact	MVP Scope Details
----	------	-------------	----------	------------	--------------	--------	-------------------

PD-2.1	Velocity Degradation Calculator	Compare real-time cycle times, ticket transit velocities, and active blockers against the project baseline to flag statistical deviations.	P0	High	PMO, COO	Critical: Automatically tracks changes in historical velocity.	Calculate a rolling 7-day velocity metric updated daily for every managed Jira Project.
PD-2.2	Communication Sentiment Decay Tracker	Run lightweight NLP models to analyze sentiment trends, friction, and anxiety levels across team channels.	P1	High	PMO, COO	Medium: Flags team-level burn-out or deep-seated technical frustrations.	Generate a normalized, anonymized channel Sentiment Score (0.0 to 1.0) daily. Never expose individual employee scores to preserve privacy.
PD-2.3	Dynamic Dependency Risk Engine	Map and analyze inter-team dependencies by scanning commit patterns, issue links, and active discussions (e.g., Team B waiting for Team A to release an API).	P0	High	PMO	High: Exposes silent path dependencies that standard Gantt charts miss.	Automatically tag and rank dependency nodes on the dashboard when critical keywords indicative of blocked workflows (e.g., "waiting on", "depends on", "blocked by") are detected in communication patterns.
PD-2.4	Project Drift Score Computation	Produce an aggregated 'Drift Coefficient' ($SD_c \times 100\%$) indicating the likelihood of project delay.	P0	High	PMO, COO	Critical: Serves as the primary operational trigger for warning-level dashboards and automation loops.	Utilize a weighted scoring formula combining ticket velocity drift (40%), active dependency blockers (40%), and collaboration alignment drop-offs (20%).

5.3 Multi-Agent Coordination Orchestrator (MA)

The autonomous engine that drives self-healing workflows. Instead of standard, linear rules, it uses a cooperative team of LLM agents designed to coordinate actions, update tasks, and gather input directly from team members.



ID	Name	Description	Priority	Complexity	Target Users	Impact	MVP Scope Details
----	------	-------------	----------	------------	--------------	--------	-------------------

MA-3.1	LangGraph Orchestration Engine	A FastAPI execution module running parallel LLM agents utilizing specific roles and communication protocols.	P0	High	All	Critical: Coordinates complex workflow investigations.	Supports three default agent profiles: Triage Agent (interrogates context), Coordination Agent (drafts communications and actions), and Governance Agent (verifies rules and constraints).
MA-3.2	Contextual Memory Engine	Manage agent memory cascades, ensuring information gathered during discussions is cached in Redis and written back to the CKG.	P0	High	All	High: Ensures agents don't repeat questions or lose context during extended multi-hour workflows.	Store dynamic conversation state variables inside Redis with a strictly configured 72-hour time-to-live (TTL).
MA-3.3	Human-in-the-Loop Safeguard	Ensure all high-impact actions (e.g., rewriting dates, assigning tickets, broad-channel messaging) require explicit PM approval before dispatch.	P0	Medium	PMO, Admin	Critical: Ensures trust and prevents erratic or loop-prone AI behavior.	Provide an "Approval Request" card in the Executive Command Center that details the target platform, the proposed change, and the context before execution.
MA-3.4	Bi-Directional Action Executor	Dispatch automated updates back to external platforms (updating status tags in Jira, assigning GitHub PR reviewers, posting Slack summaries).	P0	High	All	High: Provides the direct "self-healing" utility of the workspace.	Implement secured, token-validated API update modules for the core supported stack (Slack, Jira, GitHub).

5.4 Executive Command Center Dashboard (CC)

The interactive control workspace where operators monitor system activity, evaluate risk areas, manage ongoing agent threads, and review automation impacts.

ID	Name	Description	Priority	Complexity	Target Users	Impact	MVP Scope Details
CC-4.1	Operational Health Map	A high-fidelity visual workspace displaying current initiatives, progress paths, and their calculated Drift Coefficients.	P0	Medium	COO, PMO	Critical: Gives leaders an immediate, centralized look at delivery bottlenecks.	Built with Tailwind and React. Leverages custom search filters and dynamic status groupings.
CC-4.2	Interactive Cognitive Graph Visualizer	A network topology map showing linked systems, communication paths, and emerging operational bottlenecks.	P1	High	COO, PMO	Medium: Allows deep-dive visual exploration of cross-team dependencies.	Render node-link graphs with a dynamic UI framework, highlighting critical bottlenecks in red and agent interventions in green.
CC-4.3	Multi-Agent Execution Log	A live, diagnostic feed showing active agent activities, decision loops, human approval queries, and completed tasks.	P0	Medium	PMO, Admin	High: Provides operational transparency, letting teams see exactly what the agents are processing.	Present a detailed chronological view of agent thought-process flows, target entities, and pending operations.
CC-4.4	Low-Code Workspace Designer	A visual workspace canvas where users edit agent parameters, update alert thresholds, toggle auto-run triggers, and manage messaging styles.	P1	Medium	Admin, PMO	Medium: Empowers non-technical users to customize automation behavior without code changes.	Visual, node-based workspace built in React to define basic target conditions (e.g., "If drift coefficient >60%, trigger Triage Agent").

5.5 Enterprise Compliance & PII Masking Gateway (CG)

A high-performance security layer that guarantees zero leakage of enterprise secrets, proprietary customer records, or customer PII to external Large Language Model APIs.

ID	Name	Description	Priority	Complexity	Target Users	Impact	MVP Scope Details
CG-5.1	High-Throughput Tokenizer & Masker	Intercept outgoing payloads to detect and strip PII (names, emails, phones, SSNs) and enterprise IP secrets before forwarding data to LLM engines.	P0	High	CIO, Admin	Critical: Mandatory for enterprise-grade security compliance.	Combines deterministic, fast-regex routines with an optimized local SpaCy NER pipeline to run under 40ms.
CG-5.2	Deterministic Token Restoration Layer	Securely map masked tokens back to their original values upon receiving the final processed output from the LLM.	P0	Medium	Admin	Critical: Reconstructs contextually accurate text for delivery to end-users.	Leverages a transient, in-memory Redis lookup table storing masked mappings during LLM execution sessions.
CG-5.3	Role-Based Data Masking Engine	Ensure that data parsed into the CKG honors strict, original source permission structures (e.g., a user cannot view insights derived from Slack channels they lack access to).	P0	Medium	CIO, Admin	Critical: Prevents internal privilege escalation attacks.	Replicate target platform ACLs directly inside the CKG query generator layer.

6. Non-Functional Requirements (NFRs)

6.1 Performance & Latency

- ****Ingestion Pipeline Throughput:**** The platform must ingest, sanitize, and index up to 10,000 distinct events (Slack messages, ticket changes, Git events) per minute per tenant without backpressure latency exceeding 5 seconds.
- ****Command Center UI Load Times:**** The central workspace landing page must load in under 1.5 seconds under standard enterprise loads.
- ****LLM Gateway Processing Overhead:**** The compliance and masking gateway must add less than 50ms of overhead latency to LLM round-trips.
- ****GraphQL / SQL Queries:**** CKG semantic similarity queries using `pgvector` must return results within 200ms on a database sizing of up to 50 million entity vectors.

6.2 Security & Compliance

- ****Data Isolation:**** Multi-tenant architecture must guarantee strict physical or logical separation of data at the database level. No shared connection pools or caching structures across distinct clients.
- ****SOC2 & ISO Certification Ready:**** System logs, administrative actions, API changes, and access permissions must be completely auditable to support SOC2 Type II compliance reviews.
- ****Data-at-Rest and In-Transit Encryption:**** Enforce TLS 1.3 for all data moving over external or internal networks, and utilize AES-256 for all stored database records.
- ****No Model Training Permissions:**** Integrations with model providers (e.g., OpenAI, Anthropic, Cohere) must enforce strict contracts ensuring no user data is cached, reviewed, or used for model fine-tuning.

6.3 Reliability & Availability

- ****Target Uptime:**** The operational engine must achieve 99.95% platform availability (excluding scheduled maintenance windows).
- ****Fail-Safe Agent Termination:**** The multi-agent orchestrator must implement deterministic execution limits. No agent session can execute more than 8 sequential tool actions or run longer than 180 seconds, mitigating expensive infinite-loop traps.
- ****Platform Rate-Limit Resiliency:**** Gracefully handle API rate limits from connected systems (e.g., GitHub, Jira, Slack) utilizing back-off configurations and Redis queue scheduling.

6.4 Scalability

- ****Database Capacity:**** The system database design must easily scale past 100 million relational nodes and associated vectors within a single tenant schema without degradation of query performance.
- ****Auto-Scaling Architecture:**** Run backend services on dockerized AWS ECS using AWS Fargate, automatically scaling resources outward during high-traffic shifts (e.g., standard morning updates or end-of-quarter delivery cycles).

7. Out-of-Scope Items for Phase 1 MVP

To ensure high engineering velocity and focus on core value propositions, the following features are deferred to Phase 2:

- 1. **Native Mobile Applications:** The MVP will target responsive web layouts optimized for desktop, tablet, and mobile browsers rather than publishing native iOS or Android apps.
- 2. **Support for On-Premise Jira / Confluence Servers:** The MVP will exclusively interface with modern cloud APIs (e.g., Jira Cloud, GitHub Enterprise Cloud). Direct support for custom, legacy on-prem setups is deferred.
- 3. **Autonomous Code Merges:** The Multi-Agent platform will not autonomously merge pull requests or deploy pipeline code. It will focus on workflow coordination, dependency logging, and status tracking.
- 4. **Custom Model Fine-Tuning Interfaces:** The low-code workspace will not allow users to fine-tune custom private LLM weights. It will focus on agent workflow paths, threshold parameters, and system prompts.

8. Success Metrics & Key Performance Indicators (KPIs)

The platform's success will be tracked across three primary evaluation frameworks: Product Adoption, Product Utility, and Engineering Performance.

+-----+ +-----+			
SUCCESS METRIC EVALUATION FRAMEWORK			
+-----+			
 v		 v	
+-----+		+-----+	
Product Adoption & Growth		Operational Utility	
+-----+		+-----+	
* Weekly Active User (WAU) engagement of > 45% (target personas)		* Average Drift Coefficient Reduction of > 30%	
* Active Workspace Integration rate (90% configured within 48h)		* Direct reduction in PM status-chasing time by 60%	
* Growth rate of system queries inside Executive Dashboard		* Multi-Agent interaction approval rate of > 85%	
+-----+		+-----+	
 v			
+-----+			
Technical Platform Performance			
+-----+			
* API ingestion webhook processing latency maintained under 100ms			
* Near-zero leakage rate on continuous PII / Secrets scanning validations			
* 99.95% system execution uptime on core platform services and multi-agent loops			
+-----+			

8.1 Product Adoption Metrics

- **Target WAU Engagement:** At least 45% of targeted enterprise users (PMO Leads, Engineering Directors, Operations Managers) logging into the platform weekly.
- **Integration Activation Rate:** >90% of installed accounts completing complete verification loops across Slack, Teams, Jira, and GitHub within 48 hours of onboarding.
- **Interaction Volume:** Month-over-month expansion in the volume of semantic queries run inside the Executive Dashboard.

8.2 Product Utility & Business Value KPIs

- **Delay Warning Performance:** Mitigate delivery risks by flagging critical project delays at least 10 days before milestone completion with an accuracy rate >80%.
- **Direct Time-to-Resolution (TTR) Reduction:** Accelerate resolution on cross-team blockers by at least 45% using coordinated agent updates compared to manual, human-driven slack cycles.
- **Direct Productivity Salvage:** Save PMs and Scrum Masters an average of 8 hours per week by automating manual update collection and status check routines.
- **Agent Trust Score:** Maintain an active human acceptance and approval rate of >85% for agent-suggested actions and automated sync messages.

8.3 Platform & Architectural Telemetry

- **Zero Leakage Incidents:** Zero security events indicating unmasked PII, proprietary code, or organizational secrets reached external LLM models.
- **Data Cycle Refresh Rates:** Ensure changes to Jira tickets or GitHub PR metadata are reflected across the command dashboard in under 45 seconds.

9. Technical Appendix & Data Schema Concepts

This section provides structural guidance on how key platform components organize data to enable real-time semantic tracking and multi-agent planning.

9.1 Cognitive Knowledge Graph Core Schema Entity-Relation Concept

The Cognitive Knowledge Graph represents relationship weights that dynamically change based on continuous analysis of system activity.

```

-- Schema concept for relational + semantic persistence in PostgreSQL + pgvector

-- Enable the pgvector extension to support high-performance semantic retrieval
CREATE EXTENSION IF NOT EXISTS vector;

-- Base Node table representing entities across all connected tools
CREATE TABLE ckg_nodes (
    node_id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    external_id VARCHAR(255) UNIQUE NOT NULL, -- e.g. "JIRA-1049", "GITHUB-PR-12"
    node_type VARCHAR(50) NOT NULL,          -- e.g. "EMPLOYEE", "TASK", "REPOSITORY", "DISCUSSION_ROOM"
    display_name VARCHAR(255) NOT NULL,
    metadata JSONB DEFAULT '{}'::jsonb,      -- Stores dynamic, platform-specific attributes
    created_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP,
    updated_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP
);

-- Node Content Embeddings table for semantic similarity lookups
CREATE TABLE ckg_node_embeddings (
    embedding_id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    node_id UUID REFERENCES ckg_nodes(node_id) ON DELETE CASCADE,
    raw_content_source TEXT NOT NULL,         -- Unstructured text description before vectorization
    vector_embedding vector(1536) NOT NULL,   -- 1536-dimension embeddings generated from content
    created_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP
);

-- Edges tracking relational associations, dynamically weighted by system activity
CREATE TABLE ckg_edges (
    edge_id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    source_node_id UUID REFERENCES ckg_nodes(node_id) ON DELETE CASCADE,
    target_node_id UUID REFERENCES ckg_nodes(node_id) ON DELETE CASCADE,
    relationship_type VARCHAR(100) NOT NULL, -- e.g. "BLOCKED_BY", "ASSIGNED_TO", "COMMITTED_IN", "DISCUSSED_IN"
    interaction_weight NUMERIC(5, 2) DEFAULT 1.00, -- Dynamic strength based on frequency/sentiment
    first_detected TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP,
    last_active TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP,
    CONSTRAINT unique_source_target_rel UNIQUE (source_node_id, target_node_id, relationship_type)
);

-- Indexing vectors to enable low-latency semantic similarity search (using HNSW)
CREATE INDEX ckg_vector_hnsw_idx ON ckg_node_embeddings
USING hnsw (vector_embedding vector_cosine_ops);

```

9.2 Compliance Gateway Tokenization Flow Example

To understand how the **Enterprise Compliance Gateway (CG)** protects data, review this typical transformation sequence.

1. Incoming Context Payload from Jira Ingestion Worker:

```
> "Hi Sarah Jenkins, please ensure client account ID #US-98312 and our internal AWS key AKIAIOSFODNN7EXAMPLE are purged before you commit to the main GitHub repository."
```

2. Transformed Payload leaving the Gateway to the LLM:

> *"Hi `[PERSON\_NAME\_1]`, please ensure client account ID `[ACCOUNT\_IDENTIFIER]` and our internal AWS key `[AWS\_SECRET\_KEY\_REDACTED]` are purged before you commit to the main GitHub repository."*

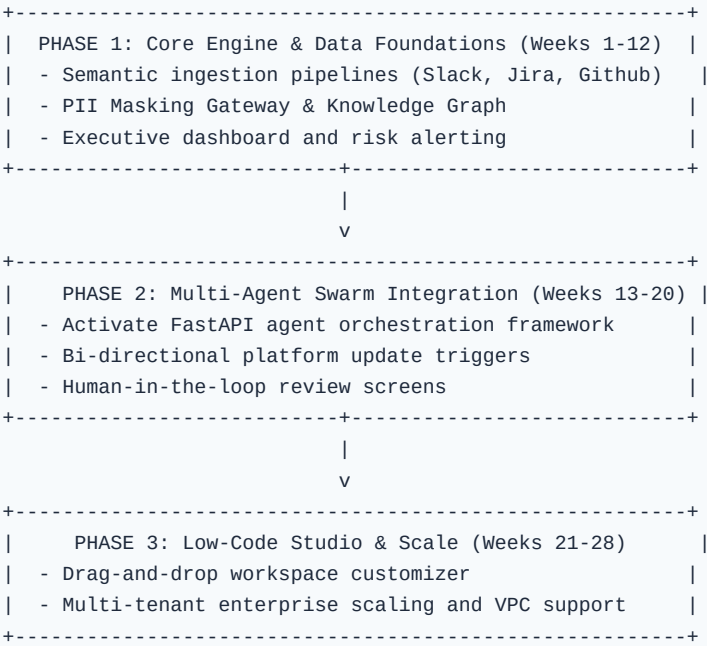
3. Execution Session Mapping cached inside L1 Redis:

```
{
  "session_id": "sess_89432_llm_call",
  "mappings": {
    "[PERSON_NAME_1]": "Sarah Jenkins",
    "[ACCOUNT_IDENTIFIER]": "US-98312",
    "[AWS_SECRET_KEY_REDACTED]": "AKIAIOSFODNN7EXAMPLE"
  },
  "ttl": 300
}
```

This mapping structure guarantees that the external model never processes proprietary values, while allowing the return payload to be cleanly reconstructed with the original parameters before being rendered to the user.

10. Phase-Wise Release Roadmap

To manage project risks, AetherOS will roll out in three phases:



- ****Phase 1 (Weeks 1-12): Foundations & Data Ingestion Engine****
- Complete basic platform API Gateway and Database schema deployments.
- Deploy the Compliance & PII Masking Gateway.
- Build out core ingestion integration connectors for Jira and Slack.
- Establish initial vector generation and the Executive Command Dashboard.
- ****Phase 2 (Weeks 13-20): Multi-Agent Orchestration & Core Analytics****
- Activate the FastAPI Agent Swarm framework.
- Deploy the Predictive Delay and Risk engine calculating Drift Coefficients.
- Launch cross-platform bi-directional action tools with PM approval gates.
- ****Phase 3 (Weeks 21-28): Custom Workspaces & Scale Operations****
- Deliver the low-code Workspace Designer dashboard.
- Optimize systems to support high-throughput load and database sizes.
- Perform pilot evaluations with three selected early enterprise partners.

02. SPRINT BACKLOG & USER STORIES

SPRINT 1: SECURE DATA INGESTION AND SEMANTIC GRAPH FOUNDATION 18 SP Total

Sprint Focus: Establish secure enterprise pipelines, PII masking middleware, and pgvector knowledge graph representation.

STORY-101 Secure PII Masking and Enterprise Compliance Gateway

5 Story Points

As a As an Enterprise Chief Information Officer (CIO), I want to I want a secure, real-time middleware layer to redact PII and enforce role-based access before data leaves our perimeter, so that so that sensitive customer information and employee identifiers are never exposed to external LLMs..

Priority: High

ACCEPTANCE CRITERIA

- ☐ All incoming communication payloads must be parsed for patterns matching SSN, emails, names, and phone numbers.
- ☐ The redaction latency must remain under 50ms per transaction.
- ☐ Masked tokens must be reversibly mapped locally in a secure cache using AES-256 for response reconstitution.
- ☐ Role-based authorization checks must be enforced using standard OAuth 2.0 scopes.

ARCHITECT GUIDANCE NOTES

Will be implemented as an Express.js middleware component running on AWS ECS Fargate, paired with Redis for rapid temporary token lookup.

STORY-102 Multi-Source Cognitive Ingestion Engine

8 Story Points

As a As a PMO Director, I want to I want continuous synchronization with Slack channels, Microsoft Teams, Jira boards, and GitHub repositories, so that so that our digital operational footprint is systematically indexed without manual tracking overhead..

Priority: High

ACCEPTANCE CRITERIA

- ☐ Establish functional Webhook listeners and polling runners for Slack, Teams, Jira API, and GitHub Webhooks.
- ☐ Handle connection rate-limiting with exponential backoff.
- ☐ Convert unstructured payloads into a standardized corporate event schema.
- ☐ Queue raw payloads safely in Redis for downstream cognitive mapping.

ARCHITECT GUIDANCE NOTES

Utilizes Redis BullMQ for reliable event-driven delivery queues before semantic embedding.

STORY-103 PostgreSQL pgvector Semantic Mapping and Knowledge Graph

5 Story Points

As a As a Systems Architect, I want to I want to transform enterprise events into vector embeddings and store them within a PostgreSQL schema using pgvector, so that so that semantic connections between disparate conversations, issues, and code changes can be instantly queried..

Priority: High

ACCEPTANCE CRITERIA

- ☐ Deploy PostgreSQL instance with pgvector extension enabled on AWS RDS.
- ☐ Define schema for organizations, projects, teams, tasks, nodes, and semantic edges.

- ☐ Configure vector column with HNSW indexes for high-speed cosine similarity searches.
- ☐ Write Python helper scripts in FastAPI to generate embeddings using OpenAI text-embedding-3-small.

ARCHITECT GUIDANCE NOTES

Ensure relational mapping links back to Jira IDs, Slack Message IDs, and GitHub Commit hashes for reference.

SPRINT 2: PREDICTIVE DELAY AND RISK ANALYTICS ENGINE

13 SP Total

Sprint Focus: Incorporate ML-driven risk scores, timeline forecasting, and communication sentiment monitoring.

STORY-201 ML-Driven Delivery Delay and Risk Predictor

8 Story Points

As a As an Enterprise Chief Operating Officer (COO), I want to I want an analytical engine that continuously correlates task velocity with team communication density, so that so that delivery bottlenecks and critical path slippages are flagged days before they occur..

Priority: High

ACCEPTANCE CRITERIA

- ☐ Develop predictive algorithm utilizing historical cycle times and current sprint burndown velocities.
- ☐ Flag projects where communication patterns indicate high circular dependencies or low active updates.
- ☐ Generate a delay probability score (0.0 to 1.0) for every active epic.
- ☐ Expose risk scores via secured REST API endpoints in FastAPI.

ARCHITECT GUIDANCE NOTES

Calculated using a hybrid model of traditional PM statistics (PERT) combined with graph network centrality heuristics.

STORY-202 Real-time Contextual Sentiment and Friction Analyzer

5 Story Points

As a As a Cross-functional Product Leader, I want to I want a natural language processing module that detects operational friction and conversational alignment shifts, so that so that team burnout, misaligned priorities, and system gaps are highlighted objectively..

Priority: Medium

ACCEPTANCE CRITERIA

- ☐ Process ingested communications (excluding redacted PII) through local sentiment classifier.
- ☐ Map sentiment trends over a rolling 7-day window to detect sustained escalation spikes.
- ☐ Categorize sentiment patterns into operational themes such as tech-debt, blocked-dependencies, or resource-constraints.
- ☐ Expose aggregated metrics that do not breach individual team-member privacy levels.

ARCHITECT GUIDANCE NOTES

Calculated at aggregated team/channel level to respect psychological safety metrics.

SPRINT 3: MULTI-AGENT COORDINATION ORCHESTRATION

13 SP Total

Sprint Focus: Build FastAPI-based agent workflows, task queue execution, and cross-silo action automation.

STORY-301 **FastAPI Multi-Agent Orchestration Framework**

8 Story Points

As a As a Systems Architect, I want to I want a FastAPI-based multi-agent supervisor pattern that spawns dedicated sub-agents for specialized tasks, so that so that complex cross-functional workflows can be handled by collaborative, specialized LLM entities..

Priority: High

ACCEPTANCE CRITERIA

- ☐ Implement a Supervisor-Agent pattern capable of decomposing complex user directives into sub-tasks.
- ☐ Establish an agent communication protocol over Redis Pub/Sub for synchronous message exchange.
- ☐ Provide persistent execution memory/state using standard session management.
- ☐ Deliver fallback strategies when an individual sub-agent fails to complete an assignment.

ARCHITECT GUIDANCE NOTES

Using LangGraph or lightweight custom state machine inside FastAPI for deterministic agent-to-agent loops.

STORY-302 **Autonomous Cross-Silo Ticket and Artifact Synchronizer**

5 Story Points

As a As a Project Management Office (PMO) Director, I want to I want autonomous agents to automatically reconcile context across Slack resolutions, code commits, and Jira issues, so that so that developers and managers do not spend manual effort updating tickets and chasing status updates..

Priority: High

ACCEPTANCE CRITERIA

- ☐ Trigger agent execution upon detecting resolved thread status in Slack or merged PR on GitHub.
- ☐ Draft precise updates for target Jira tickets reflecting status changes and implementation details.
- ☐ Request confirmation from authorized managers via interactive Slack blocks before writing mutations to Jira.
- ☐ Log agent decision-making reasoning to the centralized database.

ARCHITECT GUIDANCE NOTES

Uses Slack Interactive Blocks and Jira API for two-way verification.

SPRINT 4: COMMAND CENTER & SYSTEM OBSERVABILITY

8 SP Total

Sprint Focus: Deliver the interactive executive React dashboard and real-time agent execution tracking logs.

STORY-401 **Executive Command Center React Dashboard**

5 Story Points

As a As an Enterprise Chief Operating Officer (COO), I want to I want a high-performance web dashboard with interactive charts mapping organizational health, bottlenecks, and delivery risks, so that so that I can make data-driven resource re-allocations instantly from a single unified workspace..

Priority: High

ACCEPTANCE CRITERIA

- ☐ Develop responsive UI in Vite, React, Tailwind CSS, and Shadcn UI components.
- ☐ Integrate interactive Recharts visualizations illustrating project delivery trajectory and predictive delay risk gradients.
- ☐ Include drill-down capabilities showing active bottleneck clusters and team sentiment averages.
- ☐ Implement real-time updates using WebSockets connection to the backend API.

ARCHITECT GUIDANCE NOTES

Ensure light and dark mode capability while optimizing components for extremely fast initial paint times.

STORY-402

Real-time Multi-Agent Trace and Execution Log Viewer

3 Story Points

As a As a CIO, I want to I want to monitor the runtime trace, logical steps, and API calls executed by autonomous agents, so that so that I maintain absolute transparency, auditing capabilities, and control over AI system interactions..

Priority: Medium

ACCEPTANCE CRITERIA

- ☐ Render live agent trace timelines showing step-by-step logic, active model configuration, and PII status.
- ☐ Provide interactive approval buttons directly in the trace log for pending write actions.
- ☐ Log system usage metrics including execution duration and token cost equivalents.
- ☐ Allow filtering of logs by workspace, project, agent ID, and execution status.

ARCHITECT GUIDANCE NOTES

Uses server-sent events (SSE) for light, high-performance streaming updates of the trace logs.

03. DEVELOPMENT ROADMAP & MILESTONES

**PHASE 1: FOUNDATION,
ENTERPRISE SECURITY, & PII
MASKING GATEWAY**

Timeline Duration: 1 - 8

MVP MILESTONE

Secure Multi-Tenant Base
Architecture & Compliant PII
Gateway Operational

CORE STRATEGIC OBJECTIVES

- Establish secure AWS ECS/Fargate VPC infrastructure and multi-tenant PostgreSQL database.
- Implement AWS Cognito with Enterprise SAML/SSO integration for seamless corporate federated identity.
- Develop the real-time PII Masking Gateway middleware to intercept, scrub, and redact sensitive client data prior to vectorization or LLM ingestion.

IMPLEMENTATION TASK CHECKLIST

- ☐ Provision secure AWS landing zones with multi-AZ VPC, subnets, NAT gateways, and ECS Fargate clusters using Terraform.
- ☐ Configure PostgreSQL database instance with pgvector extension enabled for future semantic embedding storage.
- ☐ Develop Node.js (Express.js) Core API Gateway scaffolding with OAuth2 and AWS Cognito SSO integration.
- ☐ Build the Enterprise Compliance & PII Masking Gateway module using high-performance regex libraries and named entity recognition (NER) models to detect and redact SSNs, emails, names, and custom-defined corporate secrets.
- ☐ Construct Redis cache clusters for session management, token validation, and rate-limiting incoming API requests.

PHASE COMPLETION READINESS GATES

- ☐ Successful SAML SSO login flows validated against major identity providers (Okta, Azure AD).
- ☐ PII masking latency verified under 50ms for raw text payload blocks up to 50KB with a 99.9% detection accuracy of sensitive entities.
- ☐ Comprehensive SAST (Static Application Security Testing) scan report with zero 'Critical' or 'High' vulnerabilities.

PHASE 2: COGNITIVE INGESTION & SEMANTIC GRAPH MAPPING

Timeline Duration: 9 - 16

MVP MILESTONE

Continuous Ingestion Pipelines & Unified Knowledge
Graph Live

CORE STRATEGIC OBJECTIVES

- Construct robust integration connectors for Slack, MS Teams, Jira, GitHub, and enterprise emails.
- Design and deploy the live semantic organizational knowledge graph mapping relationships between people, projects, assets, and conversations.
- Establish real-time processing pipelines to translate unstructured communication into structural vector embeddings.

IMPLEMENTATION TASK CHECKLIST

- ☐ Write secure OAuth connector modules and Webhook receivers for Slack, Teams, Jira, and GitHub APIs with retry policies.
- ☐ Implement a Node.js-based streaming worker framework to parse incoming communication payloads asynchronously.
- ☐ Develop pipeline to split texts, generate semantic embeddings using OpenAI text-embedding models, and store them in PostgreSQL via pgvector.
- ☐ Implement entity-relationship schemas in the DB to represent metadata linkages (e.g., Slack thread linked to Jira Ticket, linked to GitHub PR, assigned to Developer).
- ☐ Perform stress-testing on the ingestion engine simulating up to 100 concurrent enterprise write events per second.

PHASE COMPLETION READINESS GATES

- ☐ Consistent, latency-resilient ingestion of unstructured data from 4 major integrations concurrently.
- ☐ Deterministic graph lookup query time of less than 200ms for 3-degree node separations on a test corpus of 1 million nodes.
- ☐ Validation that all metadata linked in pgvector preserves structural references back to the source platforms safely.

PHASE 3: PREDICTIVE DELAY, RISK ENGINE, & SENTIMENT ANALYTICS

MVP MILESTONE

ML-Driven Analytical Risk Model Deployment

Timeline Duration: 17 - 24

CORE STRATEGIC OBJECTIVES

- Build an analytical machine learning model to evaluate communication velocity, ticket drift, and historical cycles.
- Establish sentiment analysis routines that monitor organizational distress, fatigue, or operational friction safely.
- Formulate predictive risk scoring logic that flags timeline slippage with explicit statistical confidence.

IMPLEMENTATION TASK CHECKLIST

- ☐ Develop Python-based ML services within a FastAPI framework for predicting project ticket resolution slippage.
- ☐ Integrate lightweight sentiment analysis models (BERT-based or proprietary API) optimized for collaborative enterprise workspaces.
- ☐ Train risk-classification models using historical sprint metrics, task re-assignment frequencies, and idle-time features.
- ☐ Create API endpoints in FastAPI to serve risk probabilities, bottleneck indicators, and recommended mitigation actions.
- ☐ Implement Redis-backed queues to handle asynchronous batch analysis of historical workspace trends without slowing real-time ingestion.

PHASE COMPLETION READINESS GATES

- ☐ Risk classification models achieving at least 85% precision and recall on historical project bottleneck test datasets.
- ☐ Successful computation of workspace sentiment profiles without saving or leaking raw chat transcripts post-analysis.
- ☐ Prediction API response times maintained below 150ms for on-demand dashboard requests.

PHASE 4: MULTI-AGENT COORDINATION ORCHESTRATOR

MVP MILESTONE

Autonomous Multi-Agent Swarm Operational with Closed-Loop Execution

Timeline Duration: 25 - 32

CORE STRATEGIC OBJECTIVES

- Architect a multi-agent orchestration layer powered by FastAPI and LangChain/LangGraph to manage workflow routing.
- Create specialized system agents (e.g., Follow-up Agent, Ticket Sync Agent, Resource Allocator Agent).
- Develop human-in-the-loop validation mechanisms to allow supervisors to inspect, approve, or reject autonomous agent actions.

IMPLEMENTATION TASK CHECKLIST

- ☐ Design multi-agent interaction protocols with persistent memory stores utilizing PostgreSQL state tracking.
- ☐ Develop the Ticket Sync Agent to auto-draft ticket updates, re-assign subtasks, and close stale issues based on GitHub commits and Slack discussions.
- ☐ Build the Follow-up Agent to proactively prompt team members for status blockers in Slack/Teams and record natural language updates.
- ☐ Implement Redis pub/sub queues to orchestrate asynchronous agent actions and prevent race conditions or infinite feedback loops.
- ☐ Construct the Approval & Inspection API state machine requiring human authorization for high-impact actions (e.g., changing sprint dates or reallocating budgets).

PHASE COMPLETION READINESS GATES

- ☐ Agents can autonomously complete closed-loop workflows (e.g., detect developer push -> close Jira -> notify PM on Slack) with zero human intervention in test environments.
- ☐ Zero execution loops detected during a continuous 48-hour automated multi-agent simulation run.
- ☐ Human-in-the-loop dashboard state changes consistently reflect across active agent loops within 500ms.

PHASE 5: EXECUTIVE COMMAND CENTER & LOW-CODE INTERFACE

Timeline Duration: 33 - 40

MVP MILESTONE

Interactive React Dashboard and Configuration Console
Fully Functional

CORE STRATEGIC OBJECTIVES

- Develop the Executive Command Center frontend using Vite, React, Tailwind CSS, and Shadcn UI.
- Implement high-fidelity interactive data visualizations of organizational health, bottlenecks, and active agent execution paths using Recharts.
- Construct a user-friendly low-code interface for configuring agent triggers, notification thresholds, and custom integration rules.

IMPLEMENTATION TASK CHECKLIST

- ☐ Set up Vite-based React single-page application structure with Tailwind CSS and pre-configured Shadcn components.
- ☐ Build real-time dashboards utilizing Recharts for timeline projections, network collaboration graphs, and sentiment metrics over time.
- ☐ Create detailed agent performance monitoring logs showing step-by-step reasoning chains (CoT) and token consumption metrics.
- ☐ Design and code the visual low-code rules engine that enables business operations leads to drag-and-drop conditions for agent trigger mechanisms.
- ☐ Integrate the frontend application with Cognito auth flows, handling role-based routing (Admin vs. Operator vs. Viewer).

PHASE COMPLETION READINESS GATES

- ☐ Lighthouse performance scores for the React application exceed 90 across all categories (Performance, Accessibility, Best Practices).
- ☐ Real-time chart updates render smoothly under Simulated WebSocket loads of 1,000 active concurrent event broadcasts.
- ☐ Successful creation and execution of an agent workflow via the low-code visual creator interface by non-technical testers.

PHASE 6: ENTERPRISE HARDENING, SCALE, & LAUNCH

MVP MILESTONE

Timeline Duration: 41 - 48

Production Launch, SOC2 Readiness, and Multi-VPC Support

CORE STRATEGIC OBJECTIVES

- Perform comprehensive performance optimization, load testing, and caching strategy refinements across all microservices.
- Acquire SOC2 Type I readiness documentation and finalize enterprise-grade compliance measures.
- Launch pilot deployments with lighthouse enterprise clients, fine-tuning custom cognitive graph models.

IMPLEMENTATION TASK CHECKLIST

- ☐ Conduct rigorous end-to-end performance and penetration tests on API Gateway, Compliance Gateway, and FastAPI endpoints.
- ☐ Establish automated daily model validation pipelines to flag and mitigate LLM drift or degradation in translation quality.
- ☐ Formulate Terraform templates to support private VPC isolated deployments for dedicated Enterprise Tier customers.
- ☐ Gather operational telemetry, refine system instructions based on pilot user feedback, and optimize token usage to reduce runtime costs.
- ☐ Finalize platform SLAs, disaster recovery procedures, and officially shift the operational control center to public availability.

PHASE COMPLETION READINESS GATES

- ☐ Platform successfully handles a simulated load of 10,000 active concurrent users and 1,000,000 ingested communication events daily without degradation.
- ☐ All automated security testing outputs clean logs with 100% adherence to SOC2 compliance standards mapped.
- ☐ Mean Time to Detect (MTTD) operational bottlenecks verified under 10 minutes from source message ingestion to predictive alert emission.